

Mock Design: Twitter / X

Phase 4 · Interview-Ready · Estimated time: 2 hours

1. Concept From First Principles: Applying the Framework

Twitter/X is close cousin to Chapter 8's news feed, sharing the same core hybrid fan-out model — but it adds two genuinely distinct problems worth a full chapter: **timeline generation at truly extreme follower skew** (Twitter's celebrity accounts can have 100M+ followers, more extreme than a typical social app) and **trending topics**, a separate, largely independent subsystem with its own scale characteristics. Expect interviewers to explicitly want to see you distinguish this problem from a generic "design a feed" question.

Step 1 — Clarify requirements

- Functional: post short text updates (tweets); follow other users; view a home timeline of tweets from followed accounts; view trending topics; search tweets.
- Non-functional: extremely read-heavy; must gracefully handle a small number of accounts with an enormous follower count; trending topics should reflect activity within the last several minutes to hours, not be fully real-time to the second.

Step 2 — Estimate scale

Assume 250 million daily active users, ~500 million tweets/day (~5,800 writes/second average), and each user checking their timeline multiple times/day, giving a read:write ratio in the same 100:1–1000:1 range as Chapter 8. The number that matters most here specifically: Twitter/X's most-followed accounts have well over 100 million followers — meaningfully higher than a typical social app's largest accounts, which makes the celebrity fan-out problem from Chapter 8 even more pronounced here and genuinely unavoidable to address.

Step 3 — API

POST /tweets {userId, content}, GET /timeline?userId=X&cursor=Y, GET /trending?region=Y. Note the trending endpoint takes a region — trending topics are typically localized, not global, which matters for this program's India/UAE/Saudi multi-region context.

Step 4 — High-level design

The timeline half of this system reuses Chapter 8's architecture directly: Tweet Store (durable, write-once), precomputed per-user Timeline Store for the hybrid push path, and live-pull merge for celebrity accounts at read time. What's new here is a largely independent **Trending Service**, consuming the same tweet stream via a message queue but computing an entirely separate output (top topics by mention velocity), decoupled from the timeline-generation path so a slow or failing trending computation never affects the core timeline read/write path.

Step 5 — Deep dive: hybrid fan-out at extreme scale, and trending computation

Hybrid fan-out, revisited at Twitter's scale: the same push/pull split from Chapter 8 applies, but worth naming the exact mechanics of the celebrity path more precisely here given the more extreme skew. When a normal user's timeline is read, the system merges their precomputed pushed timeline with a targeted live query against just the specific celebrity accounts they follow (typically a small number even for users who follow several celebrities) — not a full re-scan of anything. This targeted merge keeps the read cheap even though celebrity accounts individually have enormous reach.

Trending topics: a streaming (not batch) computation, since trending-by-nature must reflect recent activity, not yesterday's. As tweets flow through the message queue, extract hashtags/keywords and maintain a **sliding-window frequency count** per topic (directly reusing Phase 3, Ch 7's stream processing and windowing concepts) — a topic "trends" when its mention velocity within the recent window significantly exceeds its historical baseline, not just by raw volume alone (otherwise permanently high-volume topics like generic greetings would always dominate). At true global scale, exact per-topic counting across billions of tweets is expensive; a strong answer notes that **approximate counting** structures (Phase 3, Ch 10's Count-Min Sketch) are a reasonable trade for trending specifically, since exact precision on "is this the 40,001st or 40,050th mention" doesn't materially change which topics surface as trending.

Step 6 — Trade-offs and wrap-up

Volunteer: trending topics being localized by region (India's trending list differs from Saudi Arabia's) means the sliding-window computation must be partitioned by region, not global — a direct, unprompted callback to this program's India/UAE/Saudi multi-market framing (Chapter 2) that would read well in an interview at a company actually operating across these markets.

2. Real-World Example / Case Study

Twitter's own widely-discussed historical architecture (including its well-known early "fail whale" scaling struggles before the hybrid fan-out model matured) is the canonical public case study for exactly this celebrity write-amplification problem, and its trending-topics system's use of streaming, windowed, approximate counting over exact global counts is a well-documented real engineering trade-off, not a hypothetical interview simplification.

3. Diagram

```

Timeline read (reuses Chapter 8's hybrid model):
[GET /timeline] -> merge:
    (a) precomputed pushed tweets (normal follows)
    (b) targeted live query (celebrity follows only)

Trending (separate streaming subsystem):
[Tweet Stream] -> [Extract hashtags/keywords]
                    |
                [Sliding-window frequency count]
                (Count-Min Sketch for scale, per region)
                    |
                velocity >> historical baseline?
                    |
                [Trending Topics API] (by region)

Decoupled from timeline path: trending computation
failure never affects core tweet read/write.

```

4. Trade-offs Table

Decision	Alternative	Why this choice
Hybrid push/pull for timeline (as Ch 8)	Pure push or pure pull	Extreme follower skew makes pure push especially catastrophic here
Streaming trending computation	Periodic batch trending job	Trending-by-nature requires recent-activity reflection, not stale batches
Approximate counting (Count-Min Sketch)	Exact per-topic counters	Exact precision unneeded at this cardinality/scale; saves substantial memory
Region-partitioned trending	Single global trending list	Trending topics are genuinely localized; matches real multi-market operation

5. Common Interview Questions

Q1: "How is this different from designing a generic news feed?"

Model approach: Name the two genuinely distinct additions — more extreme follower skew intensifying the celebrity fan-out problem, and a largely independent trending-topics subsystem with its own streaming/windowing requirements — rather than treating this as identical to Chapter 8.

Q2: "How would you compute trending topics without exact counting becoming a bottleneck at scale?"

Model approach: Name approximate counting (Count-Min Sketch) directly, and justify why approximate precision is an acceptable trade for this specific use case, tying back to Phase 3 Chapter 10's honest cost/benefit framing for probabilistic data structures.

Q3: "Why would a topic need to exceed its historical baseline, not just have high raw volume, to trend?"

Model approach: Explain that permanently high-volume, non-newsworthy terms would otherwise dominate trending lists forever; comparing recent velocity against a historical baseline surfaces genuine spikes instead.

6. Practice Exercises

- 1 Run the full six-step framework for "design Twitter," out loud, with a timer, explicitly naming what differs from Chapter 8's news feed.
- 2 Design the sliding-window frequency-count mechanism for trending topics: what window size, what data structure, how do you compare against a historical baseline?
- 3 Explain how you'd partition the trending computation by region (India, UAE, Saudi Arabia) without simply running three fully separate, unrelated pipelines.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"System Design Interview Walkthrough: Design Twitter"	Directly covers timeline generation and the celebrity fan-out problem at Twitter's scale.
English	"System Design for Twitter (Timeline, Live Updates, Tweeting) System Design Interview Prep"	Covers timeline and trending as related but distinct subsystems.
Hindi	"System Design Complete Course 2026" (Hindi + English, 10+ hrs)	Bilingual course covering feed/timeline systems as part of the broader curriculum.
Hindi	"Complete System Design Roadmap 2026 HLD & LLD" — CodeHelp	Hindi roadmap reinforcing streaming and windowing fundamentals this chapter builds on.

8. Key Takeaways

- Twitter/X's timeline generation reuses Chapter 8's hybrid push/pull model, intensified by more extreme follower skew.
- Trending topics is a largely independent, streaming (not batch) subsystem, decoupled from the core timeline path.
- Sliding-window frequency counting, compared against a historical baseline, is what distinguishes genuine trends from permanently high-volume terms.
- Approximate counting (Count-Min Sketch) is a reasonable, well-justified trade for trending computation at extreme scale.
- Trending topics are genuinely localized by region, which should be reflected architecturally, not just as a UI filter.
- Decoupling trending computation from the core tweet read/write path means a trending-system failure can't degrade core functionality.
- This chapter rewards explicitly distinguishing itself from a generic feed problem rather than repeating Chapter 8 unchanged.